
boolean



Data Fetching

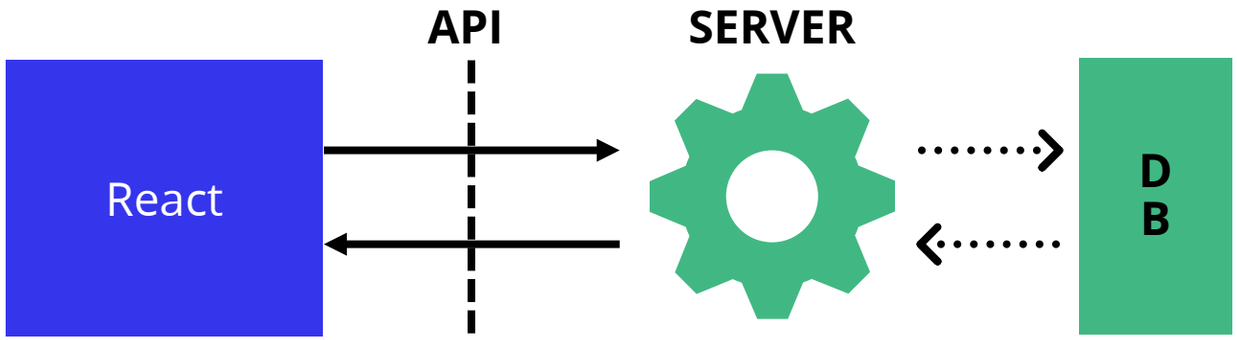
lavorare con le API 🐝

Data fetching

Lavorare con le API 🐝

Fino ad ora abbiamo lavorato solo con dati locali. Tuttavia, in un'applicazione completa, è comune che si debba interagire con servizi backend per svolgere diversi compiti, tra cui: la **persistenza dei dati** attraverso database, l'**elaborazione complessa** dei dati, l'**invio di email** e molto altro.

React, essendo una libreria JavaScript per lo sviluppo di UI, può sfruttare il **protocollo HTTP** per effettuare **richieste asincrone** verso server remoti.
Questo permette all'applicazione frontend di **ottenere** e **inviare dati al server** consentendo una comunicazione bidirezionale tra il frontend e il backend dell'applicazione.



Data fetching

Lavorare con le API 🐝 in React

In React, possiamo inviare richieste a endpoint API in diversi modi: utilizzando la funzione nativa di JavaScript **fetch** o tramite librerie di terze parti come **Axios**.

La scelta di quando effettuare queste richieste dipende dalle necessità dell'applicazione.

- **Evento dell'utente**: Possiamo effettuare una richiesta API in risposta a un evento dell'utente, come un click su un pulsante o il submit di un form.
- **Ciclo di vita del componente**: A volte è necessario effettuare richieste API in determinati momenti durante il ciclo di vita di un componente.
  Ad esempio, al momento del caricamento del componente, potremmo voler recuperare i dati iniziali necessari per renderizzare correttamente il componente.

Data fetching

Richiesta API al click su un bottone

Vogliamo mostrare una lista di todos dopo che l'utente ha cliccato su un bottone.

Per fare ciò, possiamo seguire questi step:

- **Creazione dello stato**: Creiamo una variabile di stato in cui conserveremo la lista dei "todos". Utilizziamo lo stato per aggiornare dinamicamente la UI quando otteniamo i dati dal server.
- All'**onClick** sul bottone effettuiamo la chiamata alle API
- **Gestiamo la risposta**: Quando otteniamo la risposta dal server, inseriamo i dati ricevuti nello stato creato in precedenza utilizzando la funzione **setTodos**.

```jsx
import { useState } from "react";

export default function App() {
  const [todos, setTodos] = useState([]);

  // Fetching dei dati
  async function fetchTodos() {
    const response = await fetch("https://jsonplaceholder
    const data = await response.json();
    setTodos(data.todos);
  }

  return (
    <div>
      <button onClick={fetchTodos}>Carica Todos</button>
      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>{todo.title}</li>
        ))}
      </ul>
    </div>
  );
}
```



LIVE CODING

Richiesta API al click

Data fetching

Richiesta API al caricamento del componente

Semplice vero? 😉

Ma in alcuni casi, potrebbe essere più utile effettuare la richiesta alle API automaticamente al caricamento del componente, senza richiedere l'interazione esplicita dell'utente.

Potremmo provare ad invocare la funzione **fetchTodos** subito dopo la sua dichiarazione, vediamo che succede!

```jsx
import { useState } from "react";

export default function App() {
  const [todos, setTodos] = useState([]);

  // Fetching dei dati
  async function fetchTodos() {
    const response = await fetch("https://jsonplacehold
    const data = await response.json();
    setTodos(data.todos);
  }
  fetchTodos();

  return (
    <div>
      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>{todo.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

Data fetching

3....2....1....Boom 💥



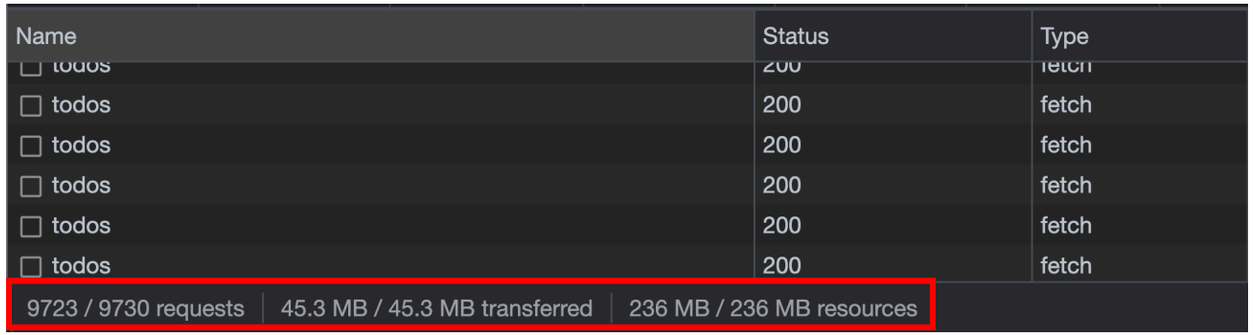
Data fetching

Richiesta API al caricamento del componente

Il codice che abbiamo scritto in precedenza ha creato un **loop infinito** di richieste! 😱

Per capire il motivo, dobbiamo tornare alla discussione su come avviene il re-render di un componente in React.

Quando uno stato all'interno del componente viene aggiornato utilizzando la funzione **setState**, React renderizza nuovamente il componente. Nel nostro caso, la chiamata API viene effettuata all'interno del componente e, quando riceviamo la risposta e aggiorniamo lo stato, viene scatenato un re-render. Tuttavia, questo re-render comporta nuovamente l'esecuzione del codice che effettua la chiamata API, creando così un ciclo infinito di richieste e re-render.



Viene in mente qualcosa in React che ci permetta di eseguire una funzione SOLO al primo rendering dei componenti?



Data fetching

useEffect: richiesta API al caricamento del componente

Possiamo utilizzare lo **useEffect!**

Basta passare **un array vuoto** come elenco delle dipendenze. 😉

```jsx
import { useEffect, useState } from "react";

export default function App() {
  const [todos, setTodos] = useState([]);

  useEffect(() => {
    // Fetching dei dati
    async function fetchTodos() {
      const response = await fetch("https://jsonplaceholder.typi
      const data = await response.json();
      setTodos(data.todos);
    }

    fetchTodos()
  }, []); // Solo al primo rendering


  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.title}</li>
      ))}
    </ul>
  );
```



LIVE CODING

Richiesta API al caricamento del componente

ESERCIZIO